



INSTITUTO DE CIÊNCIA E TECNOLOGIA
UNIVERSIDADE ESTADUAL PAULISTA

ESTRUTURA DE DADOS

SISTEMA DE ANÁLISE DE DETECÇÕES DE UVAS

Professor:

Dr. Leopoldo André Dutra
Lusquino Filho

Aluno - RA :

Igor C. Freire - 251270688

SOROCABA

29/06/2026

IGOR CAMINHOTO FREIRE

RELATÓRIO DE ESTRUTURA DE DADOS
SISTEMA DE ANÁLISE DE DETECÇÕES DE UVAS

SOROCABA

29/06/2026

Lista de Figuras

1	Escalabilidade do tempo de busca por estrutura.	9
2	Desempenho comparativo (Inserção, Busca e Remoção) por estrutura. . . .	10
3	Distribuição da <i>bounding area</i> antes e após a poda IQR. À esquerda (escala logarítmica), a cauda de anomalias injetadas pela R18 nas faixas de 0,38 a 0,95; à direita, a árvore AVL saneada, com a distribuição concentrada nos valores reais.	11
4	Interface CLI do sistema: menu interativo, barras de progresso e painéis de análise formatados com cores ANSI.	13

Lista de Tabelas

1	Comportamento assintótico teórico das estruturas implementadas.	6
2	Mapeamento das restrições implementadas e seus efeitos observados.	8

Conteúdo

1	Introdução	1
2	Dataset e Tratamento dos Dados	2
3	Estruturas Implementadas	3
3.1	Lista Encadeada	3
3.2	Árvore AVL	3
3.3	Tabela Hash com Encadeamento Separado	4
3.4	Tabela Hash Otimizada	4
3.5	KD-Tree	5
3.6	Bloom Filter	5
3.7	Resumo da Análise de Complexidade	6
4	Metodologia de Benchmark	7
4.1	Validade da Comparação	7
4.2	Confiabilidade das Medições	7
4.3	Restrições do Sistema	8
5	Resultados e Discussão	9
5.1	Escalabilidade	9
5.2	Desempenho por Operação Fundamental	10
5.3	Consumo de Memória e Overhead do Heap	10
5.4	Tratamento de Anomalias e Poda Estatística (Restrição R18)	11
5.5	Análise de Enquadramento e Distribuição (KD-Tree)	11
5.6	Comportamento Probabilístico do Bloom Filter	12
5.7	Interface e Visualização de Dados	12
6	Conclusão	13
7	Declaração de Uso de Inteligência Artificial	15

1 Introdução

Sistemas de monitoramento agrícola coletam uma volumosa quantidade de leituras, que precisam ser armazenadas, consultadas e atualizadas em um tempo condizente com a realidade do produtor rural. Nesse contexto, a escolha e a implementação das estruturas de dados desempenham um papel fundamental na viabilidade e na eficiência técnica do sistema.

Difícilmente uma única estrutura atende a todas as exigências de uma aplicação real. Portanto, é necessário avaliar quais abordagens devem ser utilizadas, considerando as operações demandadas (como busca por chave, ordenação, etc) e o embasamento teórico de cada modelo.

Este trabalho implementa e compara seis estruturas de dados aplicadas a uma base de detecções contendo aproximadamente 26 mil instâncias de cachos de uva, sendo essa quantidade um valor robusto e representativo, ideal para submeter os algoritmos a um cenário real de estresse computacional e avaliar sua escalabilidade. Esses dados foram obtidos por meio de visão computacional, utilizando a plataforma Roboflow com exportação em formato YOLO. Cabe destacar que todo o conjunto de dados foi coletado e anotado de forma autoral para este projeto, o que garante confiabilidade para analisar características como o tamanho das caixas delimitadoras, a densidade espacial das detecções no frame e a identificação de anomalias.

O objetivo consiste em desenvolver três estruturas clássicas da ementa, duas adicionais e uma versão otimizada, operando sob o conjunto de restrições de projeto estabelecidas. O desempenho prático em operações de inserção, busca e remoção é confrontado com as respectivas complexidades teóricas assintóticas, utilizando tempos de execução obtidos sob uma metodologia controlada de medição em benchmark. Adicionalmente, as estruturas viabilizam uma análise estatística aplicada aos parâmetros morfológicos das detecções, consolidando a utilidade analítica do sistema.

2 Dataset e Tratamento dos Dados

Os dados utilizados neste projeto vêm de um dataset próprio de visão computacional, coletado e organizado pelo autor. Cada registro representa a detecção de um cacho de uva, contendo um identificador único, as coordenadas do centro do cacho no frame e suas dimensões (largura, altura e área).

No início, essas detecções estavam espalhadas em vários arquivos e pastas, que é o padrão da plataforma Roboflow para o treinamento de modelos de visão. Para consolidá-los, foi criado um único arquivo CSV mantendo a referência de cada cacho e seus atributos.

O arquivo final ficou com pouco mais de 26 mil detecções. Essa quantidade supera o mínimo exigido pelo projeto e é suficiente para mostrar, na prática, as diferenças de desempenho entre as estruturas de dados. Além disso, o teste de escalabilidade gera dados artificiais em quantidades que vão de mil a cem mil elementos. Isso permite observar como cada operação se comporta conforme o volume de dados cresce.

O tratamento dos dados foi feito em duas partes. Primeiro, foi injetado um ruído em parte das detecções (Restrição R18) para simular erros do modelo (como bounding boxes geradas com tamanhos inconsistentes), testando a resiliência do código a essas falhas. Depois, foi criada uma função para percorrer a árvore AVL e remover esses outliers. Dois métodos de limpeza foram testados: o Z-Score e o Intervalo Interquartilico. O resultado mostrou que o Z-Score não funciona tão bem nesse caso, porque os próprios ruídos inflam o desvio-padrão, fazendo com que erros passem como se fossem valores normais. Por isso, a escolha final foi usar os limites de Tukey (IQR), que não sofrem tanta influência dos erros e conseguiram remover a grande maioria das anomalias de forma mais eficaz.

3 Estruturas Implementadas

O sistema proposto indexa a mesma base de detecções simultaneamente em seis estruturas de dados. Foram implementadas três abordagens clássicas da ementa, duas estruturas adicionais para contextos específicos e uma versão otimizada de uma estrutura da ementa.

3.1 Lista Encadeada

A lista encadeada simples é uma estrutura sequencial dinâmica, na qual cada nó armazena os dados do cacho de uva e um ponteiro apontando para o próximo elemento da lista.

Foi escolhida especificamente para servir como o pior cenário do sistema. Ao obrigar a busca por elementos estruturados de forma linear, a lista permite demonstrar na prática a degradação algorítmica exigida pelas restrições do projeto (Restrição R21).

O seu diferencial, contudo, está na simplicidade de armazenamento inicial: por permitir a inserção rápida no topo, ela pode atuar como um *buffer* de ingestão primária para dados brutos antes que sejam repassados para árvores ou tabelas.

Em termos de complexidade, a inserção no início da lista possui custo constante $O(1)$. Contudo, tanto a busca quanto a remoção apresentam complexidade $O(n)$ no caso médio e no pior caso. A remoção é necessariamente linear porque, a menos que o ponteiro exato já seja conhecido, a estrutura exige a travessia sequencial dos nós para localizar a chave.

3.2 Árvore AVL

A Árvore AVL é uma árvore binária de busca autobalanceada: a cada inserção ou remoção, ela avalia o fator de balanceamento dos nós e, quando necessário, aplica rotações para manter sua altura proporcional ao logaritmo do número de elementos.

Optou-se por essa estrutura porque uma árvore de busca comum pode degenerar rapidamente para uma lista quando os dados chegam ordenados. Como esta implementação utiliza a área do cacho de uva como chave, a degeneração ocorreria caso as detecções fossem processadas em ordem crescente ou decrescente de tamanho, levando a busca ao pior caso linear. A AVL contorna esse problema e assegura um desempenho ágil e previsível.

O seu diferencial em relação às tabelas hash reside na ordenação natural dos elementos. Por manter as detecções ordenadas pelo tamanho da área, a AVL viabiliza consultas

eficientes por faixas de valores e permitiu a aplicação direta dos filtros estatísticos (Z-Score e IQR) de maneira otimizada (Restrição R18).

Para esta estrutura, as operações de inserção, busca e remoção operam em $O(\log n)$ tanto no caso médio quanto no pior caso. O custo computacional adicional para efetuar as rotações de balanceamento é mantido como constante $O(1)$ por operação.

3.3 Tabela Hash com Encadeamento Separado

É uma tabela de dispersão clássica que emprega um vetor de ponteiros e resolve colisões encadeando os elementos para um mesmo índice em uma lista.

Foi selecionada como a estrutura primária do sistema para acesso direto aos dados. Seguindo essa linha de aplicação, quando é necessário consultar isoladamente a medição de uma detecção específica pelo seu ID, a latência de acesso imediato torna-se o fator mais crítico do algoritmo.

O diferencial do encadeamento separado é a sua resiliência a altas taxas de colisões, suportando uma carga indefinida de novos elementos sem a exigência de redimensionamento da matriz principal.

No caso médio, a inserção, a busca e a remoção operam com $O(1)$. O pior caso teórico ocorre apenas quando a função de dispersão aloca todas as chaves no mesmo índice, o que tornaria as operações $O(n)$.

3.4 Tabela Hash Otimizada

Esta estrutura distribui todos os elementos de forma sequencial em um único vetor na memória, solucionando colisões por meio de sondagem linear e lidando com remoções lógicas através de marcadores.

A estrutura foi implementada para atender ao requisito de Otimização (Requisito 7) do projeto. A escolha se fundamenta na exploração da localidade de referência na arquitetura de hardware. Ao abandonar os saltos de ponteiros do encadeamento e agrupar as instâncias em um bloco adjacente, a estrutura maximiza os acertos na memória do processador.

Um grande problema desta otimização é o consumo de RAM: a matriz exige o dobro da pré-alocação de memória se comparada aos nós dinâmicos para manter um fator de

carga bom. Em contrapartida, entrega uma velocidade de execução maior. No benchmark prático do projeto, a remoção simplificada por marcadores garantiu ganhos expressivos em comparação ao hash convencional.

Suas complexidades assintóticas acompanham o encadeamento: o caso médio dita $O(1)$ e o pior caso teórico impõe $O(n)$. Na aplicação prática, contudo, as constantes suprimidas pelo *hardware* tornam suas operações significativamente mais ágeis.

3.5 KD-Tree

A k-dimensional tree é uma árvore de particionamento binário que organiza um conjunto de pontos no espaço, alternando o eixo geométrico de corte a cada nível (X e Y, neste plano bidimensional).

A escolha estrutural foca na distribuição das detecções no frame da câmera: as coordenadas (X, Y) do centro geométrico do cacho no plano da imagem. A inserção desta estrutura permite que o software analise a distribuição espacial dos dados.

O diferencial de implementação da KD-Tree no projeto é a viabilização de análises de densidade espacial. Essa indexação permitiu o mapeamento da produtividade visual (cachos por frame) dividida por quadrantes da imagem, recurso que não se extrai eficientemente das tabelas hash ou árvores binárias tradicionais.

A sua inserção possui custo médio de $O(\log n)$, podendo degradar a $O(n)$ no pior cenário de balanceamento. Uma busca por região opera em $O(\sqrt{n} + k)$, onde k é o número de itens na faixa. Cumpre ressaltar que a operação de remoção não foi implementada nesta estrutura. Trata-se de uma métrica ausente, visto que algoritmos de deleção em KD-Trees são custosos e o foco desta árvore, no contexto estático das imagens, recai integralmente nas consultas e particionamentos.

3.6 Bloom Filter

O Bloom Filter é uma estrutura probabilística ultracompacta que utiliza um array de bits aliado a um conjunto k de múltiplas funções hash para mapear e testar o pertencimento de um item a um grupo.

Adotou-se essa estrutura pois responde de forma quase instantânea se um ID de cacho já foi previamente processado e registrado ou não, poupando trabalho à CPU.

O diferencial do Bloom Filter é sua eficiência extrema no consumo de memória, indo milhares de detecções utilizando apenas uma fração de memória. O prejuízo dessa compactação é a possibilidade de registrar falsos positivos em limites estreitos. Falsos negativos, porém, são impossíveis pelo seu desenho matemático.

As operações de inserção e busca têm complexidade de $O(k)$, independentemente da quantidade de elementos armazenados. Destaca-se que o Bloom Filter não suporta a operação de remoção por definição arquitetural (alterar um bit do estado 1 para o estado 0 poderia apagar o rastro funcional de outros elementos que compartilham a colisão da função hash). Isso justifica a total ausência desta função, resultando no comportamento esperado de apontar a falsa presença de nós que foram removidos do sistema.

3.7 Resumo da Análise de Complexidade

Para fins de consolidação, a Tabela 1 resume o comportamento teórico das implementações e as particularidades de cada estrutura utilizada neste projeto.

Tabela 1: Comportamento assintótico teórico das estruturas implementadas.

Estrutura	Inserção	Busca	Remoção	Observação
Lista Encadeada	$O(1)$ (início)	$O(n)$	$O(n)$	<i>Baseline</i> ; força pior caso de busca (R21)
Árvore AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	Médio = Pior caso (autobalanceada)
Tabela Hash	$O(1)$ méd / $O(n)$ pior	$O(1)$ méd / $O(n)$ pior	$O(1)$ méd / $O(n)$ pior	Pior caso ocorre se as chaves colidirem
Tabela Hash Otimizada	$O(1)$ méd / $O(n)$ pior	$O(1)$ méd / $O(n)$ pior	$O(1)$ médio	Remoção otimizada via marcadores
KD-Tree	$O(\log n)$ méd / $O(n)$ pior	$O(\sqrt{n} + k)$	<i>Não impl.</i>	Foco exclusivo em busca espacial (vizinhança)
Bloom Filter	$O(k)$	$O(k)$	<i>Não sup.</i>	$k = n^0$ funções hash constantes

4 Metodologia de Benchmark

Para avaliar o desempenho das estruturas implementadas, cada operação fundamental foi submetida a medições empíricas controladas sobre a base real de detecções. Foram registrados o tempo de inserção, busca e remoção, o consumo de memória RAM, a taxa de colisões nas tabelas de dispersão, a latência de operações combinadas e o comportamento de escalabilidade (variando a carga de 1.000 a 100.000 elementos). Os resultados obtidos em tempo de execução foram exportados em formato CSV para facilitar a geração de gráficos e análises.

4.1 Validade da Comparação

Comparar estruturas de dados projetadas com finalidades distintas é metodologicamente incorreto. Por este motivo, as medições foram separadas em diferentes grupos de análise.

O primeiro reúne as estruturas que respondem à mesma pergunta lógica (como, por exemplo, achar um registro específico): nesse caso, somente entre a lista encadeada, a árvore balanceada AVL e as tabelas hash a comparação direta de tempo de busca é legítima.

Já o segundo isola as estruturas que respondem a perguntas intrinsecamente diferentes. O filtro probabilístico responde se uma chave possivelmente já foi processada pelo sistema, enquanto a árvore espacial responde quais detecções estão em uma zona específica do frame. Colocá-las na mesma régua de “tempo de busca por chave” invalidaria o método. Isso justifica a ausência de certas métricas comparativas nestas estruturas, exigindo que sejam analisadas separadamente por suas funções específicas.

Cabe ressaltar que métricas de construção do sistema, como o tempo de inserção inicial e o consumo bruto de memória, são universais e comparáveis entre todas as seis estruturas. A restrição de agrupamento e a invalidade metodológica aplicam-se estritamente à semântica das operações de consulta.

4.2 Confiabilidade das Medições

Três cuidados técnicos garantiram a validade empírica e a precisão dos tempos coletados durante o benchmark.

Primeiro, adotou-se um número adaptativo de repetições em laço: milhares de iterações para operações lineares custosas ($O(n)$) e milhões para operações de custo constante ($O(1)$). Isso garantiu que o tempo total de cada medição superasse a resolução base do

relógio do sistema operacional, evitando que o sistema registre incorretamente a medição como zero.

Segundo, os valores buscados nos testes foram alternados a cada repetição. Isso impediu que o compilador aplicasse otimizações irreais durante a medição. Se o sistema buscasse sempre o mesmo elemento, o compilador poderia perceber que a operação era repetitiva e executá-la apenas uma única vez, o que geraria um tempo de execução falsamente rápido e invalidaria o benchmark.

Terceiro, a ocupação de memória foi aferida seguindo duas ideias: a estimativa teórica via `sizeof` e a medição do consumo real de memória do processo por meio de requisições diretas ao kernel do sistema operacional.

4.3 Restrições do Sistema

O projeto foi desenvolvido sob um conjunto de restrições operacionais, cobrindo diferentes categorias. A Tabela 2 mapeia cada exigência imposta, a sua categoria, a respectiva implementação técnica e o efeito prático observado na execução do código.

Tabela 2: Mapeamento das restrições implementadas e seus efeitos observados.

Restrição	Categoria	Implementação Técnica	Efeito Prático Observado
R2	Memória	Estabelecimento de um teto de alocação de nós na Árvore AVL. Novas instâncias que extrapolam a cota são barradas.	Teste isolado comprova o bloqueio de inserções ao atingir o limite exato configurado na RAM.
R9	Disponibilidade / CPU	Bloqueio condicional de rotinas de consulta caso a CPU esteja ocupada com processamento prioritário.	Tentativa de busca é negada sob simulação de carga de processamento e liberada logo após.
R11	Rede / Comunicação	Injeção de atraso periódico (delay forçado de thread) no bloco de leitura para simular a latência de transmissão.	Cadência de ingestão visivelmente mais lenta.
R18	Qualidade dos Dados	Injeção deliberada de ruído numérico extremo nas dimensões das bounding boxes em $\sim 10\%$ do dataset.	<i>Outliers</i> introduzidos com sucesso na base e posteriormente podados pela auditoria de Interquartil (IQR).
R21	Algorítmica	Preservação da lista encadeada obrigando travessia linear para localização de índices específicos.	Degradação $O(n)$ comprovada; a lista é ordens de grandeza mais lenta que a AVL e as Tabelas Hash.

5 Resultados e Discussão

Esta seção apresenta os resultados obtidos nos testes de desempenho e os confronta com o comportamento teórico esperado para cada algoritmo. Mais do que expor métricas de tempo de execução, o objetivo desta análise é interpretar as divergências entre a teoria assintótica e a prática, frequentemente atribuíveis a efeitos de hardware e às características reais do conjunto de dados processado.

5.1 Escalabilidade

O teste de escalabilidade confirma as classes de complexidade previstas para as estruturas comparáveis. Ao variar a carga de 1.000 para 100.000 elementos, o tempo de busca na Lista Encadeada cresceu de 890 ns para 288.269 ns — um aumento de aproximadamente 324 vezes —, evidenciando o seu comportamento linear $O(n)$. No mesmo intervalo, a Árvore AVL cresceu apenas ~ 4 vezes (de 7,47 ns para 27,98 ns), coerente com o comportamento logarítmico $O(\log n)$, enquanto a Tabela *Hash* permaneceu praticamente constante (de 2,25 ns para 2,54 ns), confirmando $O(1)$.

Observou-se, contudo, que o tempo da Lista Encadeada cresceu além do fator de $\sim 100\times$ teoricamente esperado. Atribui-se essa superlinearidade (aumento de $324\times$) ao efeito de *Cache Miss*: com poucos elementos, a estrutura cabe inteiramente na memória Cache L1 do processador; em larga escala, os saltos de ponteiros pela RAM geram atrasos sucessivos de busca na memória física. Trata-se de uma divergência esperada entre o modelo assintótico, que ignora constantes físicas, e a execução real do sistema.

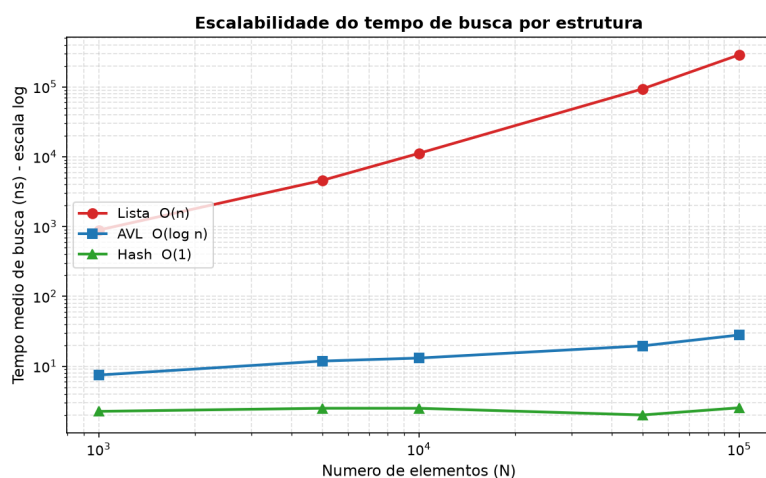


Figura 1: Escalabilidade do tempo de busca por estrutura.

5.2 Desempenho por Operação Fundamental

A comparação de latência entre as estruturas evidencia o expressivo ganho de desempenho da Tabela *Hash* Otimizada. Na operação de busca, a Hash Otimizada e a Hash com Encadeamento cravaram 1,99 ns e 2,76 ns, respectivamente, seguidas pela AVL (17,3 ns) e pela letárgica Lista Encadeada (28,97 μ s). Destaca-se que, para os 500.009 *slots* disponíveis, o fator de carga foi de apenas 4,64%, gerando ínfimas 1.841 colisões, o que explica a velocidade das dispersões.

O contraste de performance mais nítido ocorreu na operação de remoção: a estrutura otimizada operou na marca de 13,6 ns por operação, contra 87,5 ns do encadeamento tradicional. Essa diferença decorre diretamente da estratégia de deleção adotada. A tabela otimizada realiza apenas uma deleção lógica (marcação de *tombstone*), mantendo o vetor intacto. A tabela com encadeamento realiza uma deleção física, incorrendo no alto custo de acionar o gerenciador de memória do sistema operacional para desalocar o nó.

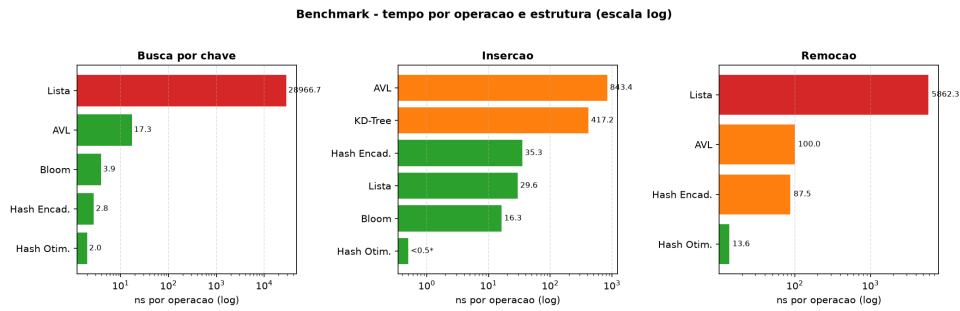


Figura 2: Desempenho comparativo (Inserção, Busca e Remoção) por estrutura.

5.3 Consumo de Memória e Overhead do Heap

Ao auditar o consumo de espaço, comparou-se a ocupação teórica via `sizeof` com o consumo real aferido pelo sistema operacional (*Working Set Size* - RSS). O resultado demonstrou que o RSS medido (60.356 KB, $\sim$ 58,9 MB) foi o dobro da estimativa estrutural parcial (29.190 KB, $\sim$ 28,5 MB).

Essa divergência é duplamente explicada. Primeiro, pela limitação da fórmula de estimativa teórica, que contabilizou apenas as matrizes das estruturas base, omitindo as dezenas de milhares de nós dinâmicos da KD-Tree e da Lista Encadeada. Segundo, pelo *overhead* exigido pelo gerenciador de *heap* do C++. A alocação dinâmica (`new`) de dezenas de milhares de nós acarreta a inclusão de cabeçalhos de controle (*metadata*) do alocador e

preenchimentos para alinhamento de *bytes*. O RSS revela esse custo oculto e estrutural, provando que estimativas algorítmicas de espaço subestimam o consumo real do processo.

5.4 Tratamento de Anomalias e Poda Estatística (Restrição R18)

A auditoria estatística revelou de forma clara a superioridade do modelo robusto sobre o modelo paramétrico tradicional. O método Z-Score sofreu severa contaminação pelos ruídos injetados (Restrição R18): os *outliers* inflaram o desvio-padrão da amostra, alargando a faixa de aceitação para o intervalo de $[0 \text{ a } 0,698]$, o que capturou apenas 1.243 anomalias.

Em contrapartida, o Intervalo Interquartil (IQR), imune à distorção de extremos, definiu uma faixa de $[0 \text{ a } 0,0329]$ e interceptou 3.141 anomalias (aproximadamente $2,5\times$ mais eficaz). Após a execução da poda estatística na Árvore AVL guiada pelo IQR, a árvore foi saneada com sucesso, restando 23.194 nós válidos e consistentes para análises futuras.

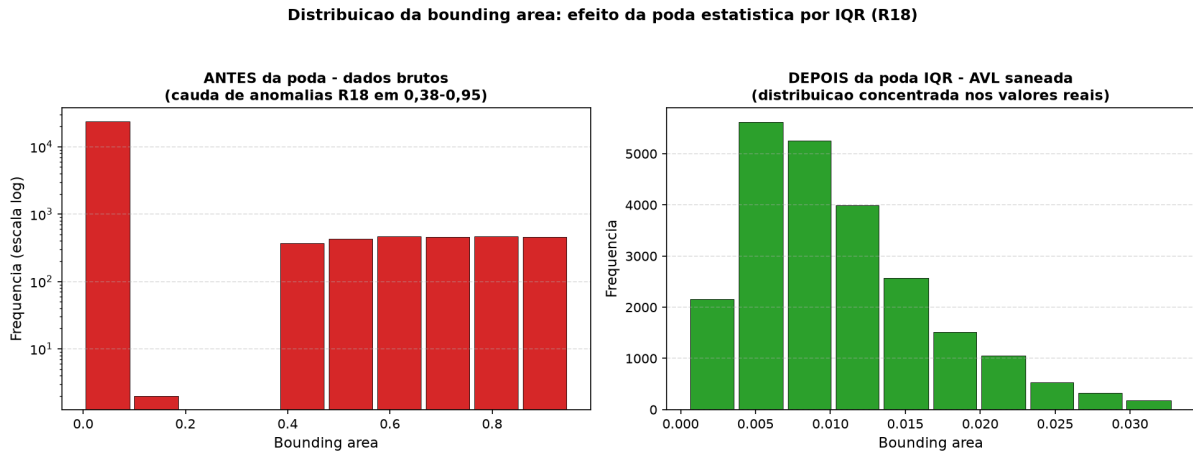


Figura 3: Distribuição da *bounding area* antes e após a poda IQR. À esquerda (escala logarítmica), a cauda de anomalias injetadas pela R18 nas faixas de 0,38 a 0,95; à direita, a árvore AVL saneada, com a distribuição concentrada nos valores reais.

5.5 Análise de Enquadramento e Distribuição (KD-Tree)

A indexação espacial fornecida pela KD-Tree provou o seu valor analítico ao calcular o comportamento visual das capturas da câmera. A estrutura mapeou um viés acentuado de enquadramento: os quadrantes superiores concentraram massivamente as detecções (10.999 no Superior Esquerdo e 11.494 no Superior Direito), contrastando fortemente com os quadrantes inferiores (1.800 e 2.042 detecções, respectivamente). Este *insight* sugere

que a câmera encontra-se angulada para captar a faixa produtiva no topo da lente, um dado relevante para a regulação do enquadramento da câmera e para a modelagem futura de redes neurais.

Além disso, a análise aglutinou as medições de produtividade, analisando os 1.842 *frames* coletados. O sistema apontou uma média de 14,3 cachos por *frame* (com pico máximo de 41). A classificação categórica identificou 338 *frames* em estado de subprodução, 1.191 na faixa ideal e 313 em sobrecarga, entregando uma métrica de valor agregado para o produtor rural.

5.6 Comportamento Probabilístico do Bloom Filter

Um dos testes validados no *benchmark* consistiu em tentar consultar a existência de um identificador removido da base de dados. Como esperado, o Bloom Filter indicou um falso positivo, afirmando que o cacho de uva ainda pertencia ao *dataset*, mesmo após ter sido extraído da AVL.

Este comportamento valida a teoria da ausência da operação de remoção nesta estrutura. Como a exclusão de um *bit* afetaria o *hash* de múltiplos elementos colidentes, o filtro probabilisticamente prefere errar afirmando que o dado existe a gerar um falso negativo, obrigando o sistema a consultar a tabela *hash* principal para a confirmação de fato.

5.7 Interface e Visualização de Dados

O requisito de interface visual (Item 10) foi plenamente atendido por meio de uma aplicação de linha de comando (CLI) robusta e formatada com caracteres de escape ANSI (Figura 4). A interface foi encapsulada com tratamento robusto de exceções nas entradas padrão (*stdin*), impedindo qualquer falha de segmentação (*crash*) caso o operador digite caracteres incorretos.

A exibição oferece *feedback* em tempo real por meio de barras de progresso adaptativas durante a ingestão massiva do *dataset*, e gera tabelas ASCII e painéis de distribuição, proporcionando uma experiência de uso intuitiva e estável.

```
=====
SISTEMA DE ANALISE DE DETECCOES DE UVA
Estruturas de Dados aplicadas a Visao Computacional - 2026
=====
>> DEMONSTRACAO: execute as opcoes 1 a 8 NESTA ORDEM <<
--- 1) Carregar e tratar os dados -----
  1. Ingerir Dataset          (R11 rede, R18 sensor)
  2. Histograma de Distribuicao (ver dados brutos)
  3. Filtrar Outliers Z/IQR   (R2 limite, R18 poda)
--- 2) Analisar -----
  4. Analise Espacial KD-Tree (produtividade por frame)
--- 3) Medir desempenho -----
  5. Teste de Escalabilidade   (+CSV p/ relatorio)
  6. Benchmark Completo        (+CSV p/ relatorio)
--- 4) Demonstrar restricoes -----
  7. Teste do Teto da AVL      (restricao R2 memoria)
  8. Teste de Bloqueio de CPU  (restricao R9)
--- Operacoes avulsas (opcional) -----
  9. Buscar registro por ID
 10. Inserir registro individual
 11. Remover registro por ID
-----
  0. Encerrar e liberar memoria
-----
Escolha: █
```

Figura 4: Interface CLI do sistema: menu interativo, barras de progresso e painéis de análise formatados com cores ANSI.

6 Conclusão

Este trabalho implementou e comparou seis estruturas de dados aplicadas a uma base real contendo aproximadamente 26 mil detecções de cachos de uvas, confrontando o desempenho prático obtido com a complexidade teórica de cada abordagem. Os experimentos conduzidos no *benchmark* confirmaram empiricamente as classes assintóticas esperadas: a estrutura linear (Lista Encadeada) degradou proporcionalmente ao volume de dados, crescendo cerca de $324\times$; a árvore balanceada (AVL) manteve um crescimento logarítmico controlado, na faixa de $\sim 4\times$; e a tabela de dispersão otimizada permaneceu com latência praticamente constante $O(1)$.

Entre os principais achados, destacou-se o compromisso (*trade-off*) entre velocidade e memória da Tabela *Hash* Otimizada, que acelerou a operação de remoção para $\sim 13,6$ ns

contra $\sim 87,5$ ns do encadeamento clássico, ao custo de exigir aproximadamente o dobro da memória pré-alocada. A análise estatística provou, de forma empírica, a superioridade do método robusto na detecção de anomalias sobre o modelo paramétrico (Z-Score), com o IQR sendo capaz de higienizar 3.141 anomalias contra apenas 1.243 do método concorrente. Adicionalmente, a indexação geométrica via KD-Tree demonstrou o altíssimo valor de uma estrutura de propósito específico, permitindo a extração de métricas de produtividade (média de 14,3 cachos por *frame*) que árvores binárias convencionais ou tabelas *hash* não conseguem suprir. O sistema também evidenciou que o modelo assintótico matemático diverge da execução real por ignorar variáveis de *hardware*, fenômeno comprovado pelo efeito de *Cache Miss* na degradação da Lista e pelo consumo efetivo de memória do processo, correspondente ao dobro da estimativa teórica do código.

Como limitações da pesquisa, reconhece-se que a simulação de algumas restrições operacionais (como a contenção de disponibilidade R9) foi tratada via controle lógico de software (*flag* de sistema) e não por uma concorrência física real na CPU. Além disso, a discrepância estrutural na medição de memória (cálculo de `sizeof` *versus* alocação real no sistema operacional) e a limitação dos dados a um único domínio visual restringem a generalização universal dos tempos de latência. Tais apontamentos, contudo, não comprometem a validade e a robustez arquitetural das conclusões alcançadas; pelo contrário, endossam a maturidade analítica do projeto.

7 Declaração de Uso de Inteligência Artificial

Declaro que ferramentas de Inteligência Artificial generativa foram utilizadas exclusivamente como apoio à codificação (depuração, otimização do *benchmark* e modularização do código) e à organização e revisão estrutural deste relatório, tendo todo o texto sido redigido pelo autor. O histórico completo das interações está disponível em: <COLAROLINKDACONVERSAAQUI>.

Referências Bibliográficas

- [1] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Algoritmos: Teoria e Prática*. 3ª ed. Rio de Janeiro: Elsevier, 2012.
- [2] BLOOM, B. H. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, v. 13, n. 7, p. 422-426, 1970.
- [3] BENTLEY, J. L. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, v. 18, n. 9, p. 509-517, 1975.
- [4] TUKEY, J. W. *Exploratory Data Analysis*. Reading: Addison-Wesley, 1977.
- [5] ROBOFLOW. *Roboflow Documentation*. Disponível em: <https://docs.roboflow.com>. Acesso em: jun. 2026.